

Аукцион выделенных серверов

Скидки до 35%



Реклама. erid: 2Vtzqw2DvtZ. АО Селектел



★ 4.45

Оценка

2195.29

Общий рейтинг

Подписаться

Selectel

IT-инфраструктура для бизнеса



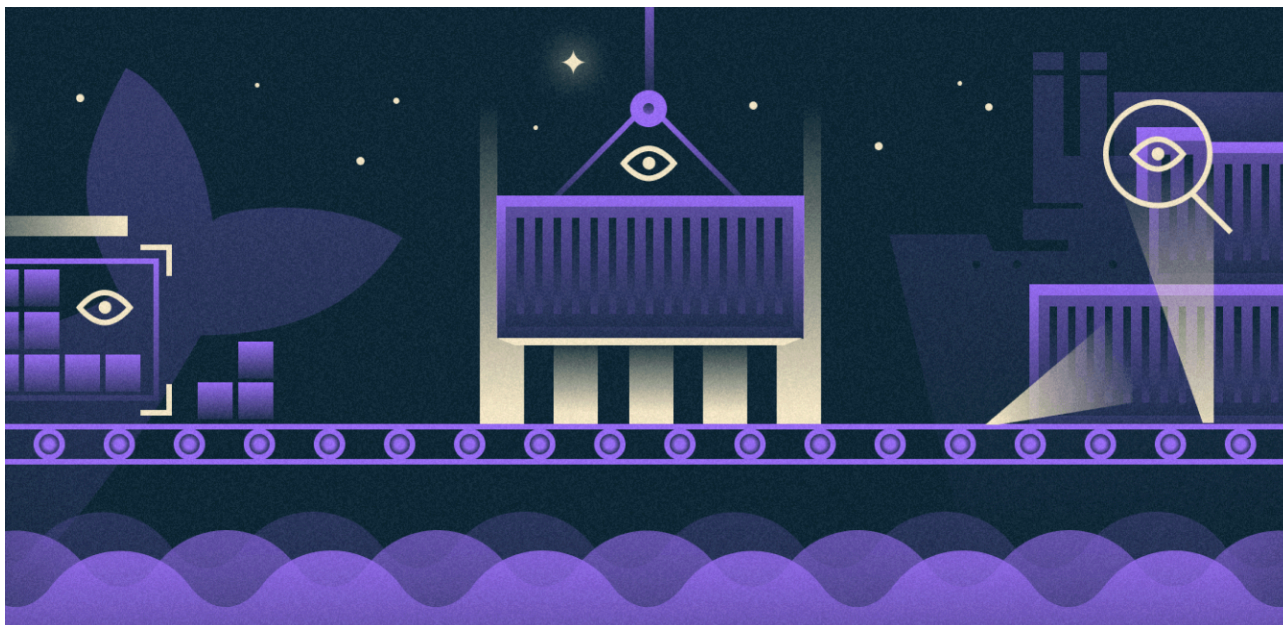
techno_mot 13 ноя 2024 в 11:42

Обзор способов защиты контейнеров Docker: от простого к сложному

🔥 Средний ⌚ 11 мин 👁 16K

Блог компании Selectel, IT-инфраструктура*, IT-компании, Open source*,
Информационная безопасность*

Обзор



Безопасность Docker — один из главных вопросов, занимающих умы DevOps-инженеров и аналитиков безопасности. Согласно последним отчетам Snyk и Red Hat более 44% всех контейнеров, которые находятся в коммерческой эксплуатации, содержат уязвимости, даже несмотря на доступность обновления базовых образов. Угроза не ограничивается выявленными недостатками — хакеры непрерывно ищут новые векторы атак на системы, построенные с использованием Docker и Kubernetes.

Используйте навигацию, если не хотите читать полностью:

- Нашумевшие инциденты 2024 года
- Виды угроз для контейнерных технологий
- Базовые методы защиты
- Продвинутое техники безопасности
- Безопасные конфигурации Dockerfile и Docker Compose

- Поиск и устранение уязвимостей
- Управление секретами и конфиденциальностью
- Краткие рекомендации по интеграции лучших практик безопасности в DevSecOps

Docker начал свое восхождение в 2014 году. Популярность контейнеров объяснить не сложно: легкость развертывания и управления, масштабирование, изоляция процессов. Как и любая технология, Docker имеет свои слабые стороны, в том числе и с точки зрения безопасности.

В статье рассмотрим методы, инструменты и практические приемы, которые помогут повысить уровень защищенности контейнеров. Начнем с основ — например, минимизации базового образа. Затем освоим более сложные средства: Snyk для сканирования уязвимости, использование секретов Docker, настройку политик безопасности.

В заключение поделимся трендами и рекомендациями, основанными на последних исследованиях, что поможет оградить контейнеры и приложения от распространенных угроз. Подскажем как снизить вероятность атак, предотвратить утечку данных и повысить устойчивость инфраструктуры.

Нашумевшие инциденты 2024 года

Февраль

Злоумышленники воспользовались «дырами» в API OpenShift и заполучили конфиденциальные сведения более чем о пятистах клиентах. Red Hat сообщила о финансовых потерях в \$5 миллионов. В ответ на атаку компания усилила меры безопасности, внедрив дополнительные проверки и обновления для своей платформы.

Март

Хакеры получили доступ к частным репозиториям пользователей через уязвимость в контейнерной технологии. В результате атаки оказалось скомпрометировано более двух тысяч образов, содержащих чувствительные данные. Docker оценила убытки в \$3 миллиона. Компания провела внутреннее расследование и улучшила свои процессы безопасности, добавив многослойную защиту и более строгие ограничения доступа.

Апрель

Представители Microsoft Azure сообщили о серьезной утечке данных из-за уязвимостей в их контейнерной инфраструктуре. Данные более миллиона пользователей были украдены. Компания оценила ущерб в более, чем \$10 миллионов и объявила о дополнительных инвестициях в безопасность своих облачных сервисов.

Июнь

Уязвимости в контейнерных приложениях GitHub привели к утечке конфиденциальной информации. Пострадало более трех тысяч проектов, что привело к значительным репутационным потерям и убыткам в \$2,5 миллиона. Руководству GitHub пришлось принимать меры по усилению безопасности и улучшению механизмов обнаружения уязвимостей.

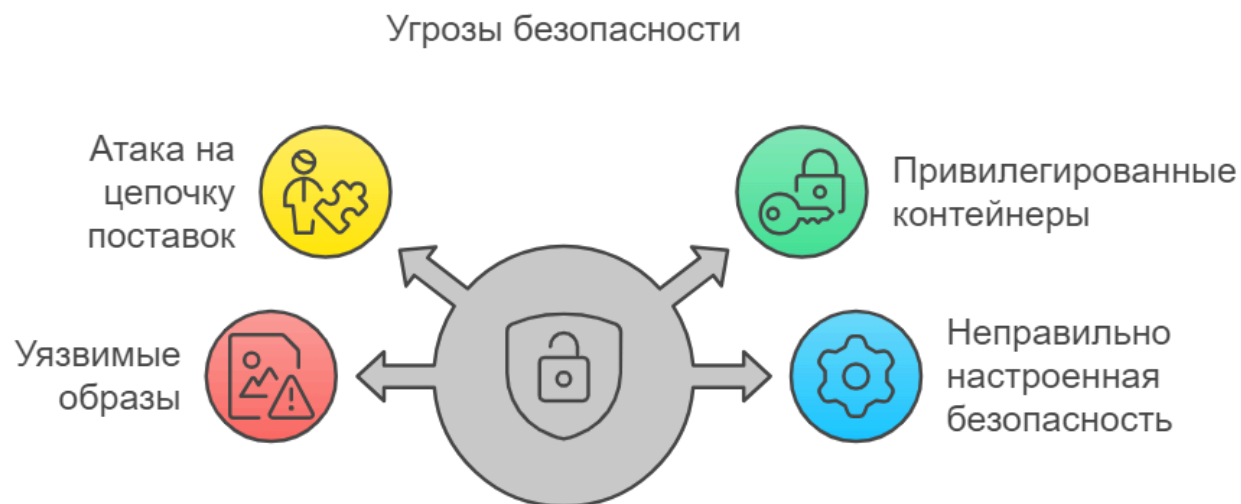
Июль

Руководство Slack сообщило о серьезной уязвимости в их контейнерной архитектуре, которая привела к утечке конфиденциальных сообщений и файлов более чем 100 000 пользователей. Уязвимость была связана с неправильной конфигурацией контейнеров, что дало возможность злоумышленникам получить доступ к

чувствительным данным. Оценка ущерба составила около \$4 миллионов. Специалисты Slack оперативно устранили уязвимость, а компания провела дополнительное обучение для разработчиков по безопасным практикам работы с контейнерами.

Виды угроз для контейнерных технологий

Контейнеры имеют многослойную структуру, что делает их идеальной мишенью для атаки. Всевозможные библиотеки и зависимости создают как правило от двух до 125 слоев, каждый из которых является дополнительным уровнем сложности. Если хотя бы один слой содержит уязвимость или вредоносное ПО, то весь контейнер перестает быть защищенным. Поскольку контейнеры часто запускаются с избыточными правами, то возникает риск эксплуатационных атак и на хостовую систему.



Атака на цепочку поставок

В последнее время атака на цепочку поставок контейнеров становится одной из наиболее серьезных угроз. Внедрение вредоносного кода происходит в публичные образы, расположенные, например, в Docker Hub путем добавления закладок в библиотеки или зависимости. Брешью чаще всего становятся уязвимости в используемых автоматизированных процессах CI/CD. В результате зараженные контейнеры начинают распространяться на все экземпляры, которые их используют, что приводит к массовым компрометациям систем.

Атака на привилегированные контейнеры

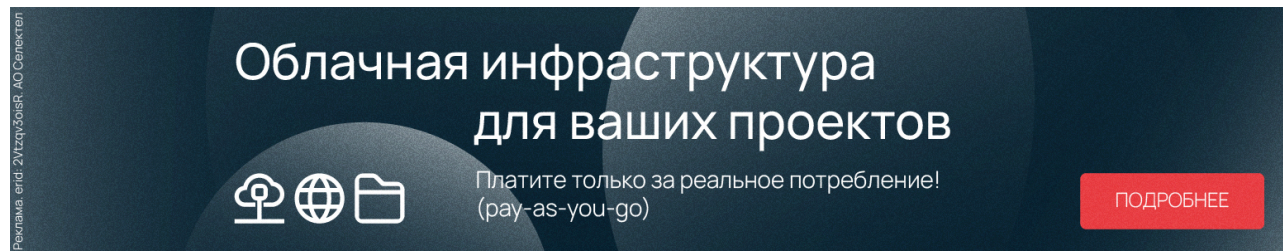
В этом виде атаки основной целью становятся контейнеры, которые запускаются повышенными привилегиями. В таких случаях атакующий для выполнения вредоносного кода может использовать уязвимости на уровне ядра хостовой системы. Например, уязвимость CVE-2023-27532 позволяла повысить права доступа внутри контейнера, что приводило к захвату контроля над всей хостовой машиной.

Угрозы, связанные с уязвимыми образами

Согласно отчету Docker за 2024 год, около 30% официальных образов в Docker Hub содержат критические уязвимости: небезопасные версии OpenSSL или устаревшие пакеты glibc. Открывается возможность для удаленного выполнения кода или утечки конфиденциальной информации. Даже несмотря на наличие современных инструментов сканирования — Trivy и Docker Scout — многие организации все еще не внедрили процесс регулярного мониторинга и обновления образов.

Эксплуатация неправильных настроек безопасности

Системные администраторы зачастую упускают важные аспекты защиты: например, ограничение привилегий и конфигурацию политик безопасности Pod Security Policies и Network Policies. Контейнеры не должны иметь возможность взаимодействовать с сетевыми компонентами. Риск утечки конфиденциальных данных (секретов) также может возникать из-за неправильно настроенного хранения.



Базовые методы защиты

Рассмотрим, как можно и нужно минимизировать риски. Здесь речь пойдет о самых простых способах.

Минимизация привилегий контейнеров



Минимизация привилегий — критически важный аспект безопасности контейнеров. Уязвимости могут возникать, так как Docker по умолчанию запускает контейнеры с базовым набором привилегий, включающим основные разрешения Linux:

- CHOWN — изменение владельца файлов и папок;
- DAC_OVERRIDE — обход ограничений доступа к файлам;
- FOWNER — управление файловыми правами;

- KILL — отправка сигналов другим процессам;
- NET_BIND_SERVICE — привязка к привилегированным сетевым портам (с номером меньше 1024);
- SETUID/SETGID — изменение идентификаторов пользователя или группы процесса.

Не все из перечисленных привилегий нужны контейнеру для выполнения задач. Параметр `--cap-drop` сбрасывает ненужные разрешения, ограничивая возможности контейнера только теми, которые действительно необходимы для его функционирования. Например, если приложение не требует доступа к низкоуровневым сетевым функциям, привилегия `NET_RAW` может быть удалена, тем самым снижая риски:

```
docker run --cap-drop=NET_RAW --cap-drop=MKNOD <image_name>
```

Всегда следует предпочитать `rootless containers`, которые запускаются от имени обычного пользователя. Это минимизирует риск того, что злоумышленник получит `root`-доступ, даже если ему удастся скомпрометировать контейнер. Использование контейнеров не требующих права суперпользователя делает атаки на уровень ядра практически невозможными и усиливает общую безопасность системы.

Изоляция и управление сетевыми политиками

`Network Policies` играют ключевую роль в безопасности контейнеров, особенно в сложных средах. Изоляция контейнеров через настройки сети помогает предотвратить несанкционированный доступ и утечку данных. Docker поддерживает изоляцию через сети, что позволяет контролировать, какие контейнеры могут

взаимодействовать друг с другом.

Kubernetes Network Policies предоставляют более детальный уровень управления трафиком между подами. Например, можно настроить политику, которая ограничивает входящий трафик только от определенных подов, создавая тем самым дополнительный уровень защиты.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-specific-traffic
spec:
  podSelector:
    matchLabels:
      app: my-app
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: allowed-app
```

В примере выше поды с меткой `app: my-app` могут получать трафик только от подов с меткой `app: allowed-app`. Очень важно внедрять ограничения на уровне сети для обеспечения безопасности контейнеризованных приложений. Применение сетевых политик и правил изоляции в Docker или Kubernetes значительно уменьшает риски и защищает приложение от атак.

Продвинутые техники безопасности

Перейдем от основополагающих способов к более сложным.

Использование AppArmor, Seccomp и SELinux

Для повышения безопасности контейнеров Docker можно привлечь ряд дополнительных инструментов, таких как AppArmor, Seccomp и SELinux. Эти технологии позволяют ограничивать действия контейнеров и улучшать общую безопасность системы.

Увеличение безопасности контейнеров с помощью AppArmor, Seccomp и SELinux



AppArmor — система управления доступом, которая позволяет задавать профили безопасности для контейнеров. Она контролирует доступность ресурсов и тем самым

помогает защитить систему от потенциально вредоносного поведения контейнеров.

Например, можно создать профиль, который разрешает контейнеру доступ только к определенным важным системным элементам, что ограничивает возможность изменять или удалять критически важные системные файлы.

Seccomp — инструмент, который позволяет ограничить набор системных вызовов, доступных контейнеру. По умолчанию Docker применяет базовые настройки Seccomp, которые блокируют системные вызовы.

Возникает вопрос: какие именно вызовы опасны и почему? Ведь такие действия, как открытие файла, чтение или запись, действительно могут привести к повреждению системы или утечке данных. Однако отключить их нельзя — работа большинства приложений будет парализована.

По умолчанию Docker с базовым профилем Seccomp блокирует следующие вызовы:

- mount — предотвращает монтирование файловых систем, защищая данные хоста от несанкционированного доступа;
- reboot — исключает возможность перезагрузки системы из контейнера;
- ptrace — блокирует инструменты отладки, которые могут дать доступ к памяти и процессам хоста;
- kexec_load — запрещает загрузку и замену ядра, что предотвращает захват системы на более глубоком уровне.

Однако разработчики могут обеспечивать дополнительный уровень безопасности и создавать собственные профили Seccomp для ограничения конкретных системных

вызовов. Например, можно отключить вызовы, которые могут привести к повреждению системы или утечке данных.

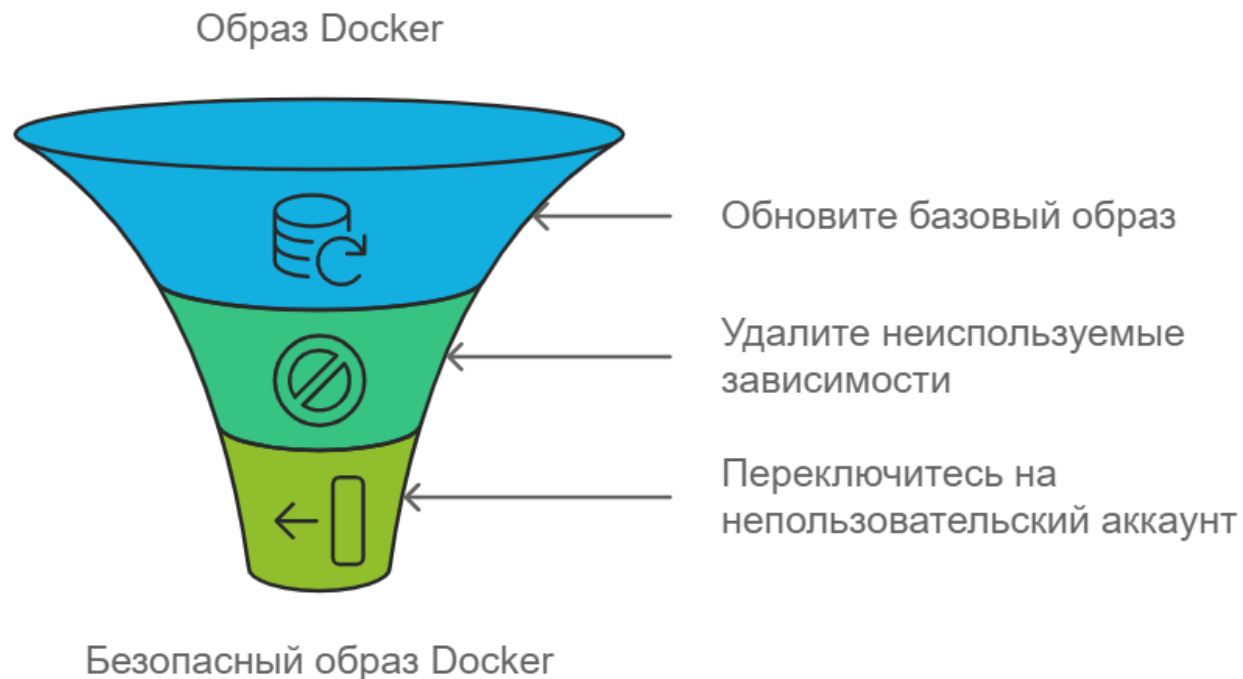
SELinux — мощная система управления, которая позволяет контролировать доступ к ресурсам на более детальном уровне. SELinux использует метки безопасности, которые применяются к объектам и процессам, позволяя управлять доступом на основе политик. В Docker можно интегрировать SELinux для повышения уровня изоляции и защиты контейнеров от атак — доступ к ресурсам на уровне операционной системы при этом будет ограничен.

Совместное использование AppArmor, Seccomp и SELinux создает многоуровневую защиту для контейнеров и значительно уменьшает риск эксплуатации уязвимостей.

Безопасные конфигурации Dockerfile и Docker Compose

При создании и развертывании контейнеров защищенность должна закладываться на этапе конфигурации. Использование безопасных практик при написании файлов Dockerfile и Docker Compose поможет избежать распространенных уязвимостей.

Улучшение безопасности образа Docker



Ниже перечислены некоторые из них.

Обновляйте базовый образ. Убедитесь, что используете актуальный базовый образ, так как устаревшие версии могут содержать уязвимости. Команда `docker pull` позволяет загрузить последнюю версию образа и тем самым защититься от известных угроз.

Удаляйте временные файлы и неиспользуемые зависимости. После установки необходимых пакетов рекомендуется очищать кэш и временные данные, например,

командой `apt-get clean && rm -rf /var/lib/apt/lists/*` . Это помогает уменьшить размер образа и удаляет потенциально уязвимые компоненты, которые могут остаться в слоях контейнера.

Запускайте контейнер от имени служебного пользователя. Для повышения безопасности рекомендуется создать служебного пользователя с ограниченными правами и запускать контейнер от его имени с помощью команды `USER`. Это предотвращает выполнение контейнера от имени `root`, ограничивая доступ к системным ресурсам и снижая риск при возможной компрометации.

Пример безопасного Dockerfile:

```
FROM python:3.9-slim
RUN apt-get update && apt-get install -y --no-install-recommends gcc
COPY . /app
WORKDIR /app
RUN pip install --no-cache-dir -r requirements.txt
USER nobody
CMD ["python", "app.py"]
```

Для повышения безопасности Docker Compose рекомендуются следующие действия:

- определите политики доступа и настройки безопасности на уровне сервиса;
- используйте ключ `security_opt` , чтобы задать параметры — такие как использование SELinux или AppArmor;
- задайте переменные окружения через файл `.env` и не храните секреты в открытом виде в `docker-compose.yml` .

Пример безопасного docker-compose.yml :

```
version: '3.7'
services:
  my_service:
    image: my_image
    deploy:
      restart_policy:
        condition: on-failure
    security_opt:
      - apparmor:my_profile
```

Поиск и устранение уязвимостей

Без регулярного сканирования безопасность может оказаться в критическом положении. Важно не только выявлять уязвимости, но и вовремя принимать меры по их устранению для минимизации рисков.

Сканирование контейнеров с помощью Trivy и Docker Scout CLI



В 2024 году наиболее популярными сканерами контейнеров, которые помимо уязвимостей определяют и некорректные конфигурации, остаются Trivy и Docker Scout CLI.

Trivy — это универсальный инструмент анализа контейнеров, который работает с Docker-образами, контейнерами, репозиториями и даже конфигурационными файлами Kubernetes. Trivy может анализировать как операционную систему, так и

зависимости приложений, выявляя критические уязвимости и ошибки конфигурации. С помощью Trivy получится встроить проверку безопасности в процесс CI/CD и автоматически анализировать образы перед их развертыванием для коммерческой эксплуатации.

Основные возможности Trivy:

- сканирование Docker-образов на наличие уязвимостей и конфигурационных ошибок;
- проверка зависимостей приложений;
- интеграция с популярными платформами CI/CD (например, GitLab CI/CD).

Роль Trivy в безопасности контейнеров



Пример команды для сканирования Docker-образа:

```
trivy image <image_name>
```

Docker Scout CLI — инструмент от разработчиков Docker, который появился сравнительно недавно и активно используется для мониторинга и повышения защищенности контейнеров. Он позволяет выявлять уязвимости в контейнерах, анализировать их зависимости и оценивать состояние безопасности образов.

Основные возможности Docker Scout CLI:

- Мониторинг и анализ уязвимостей в реальном времени;

Рекомендации по обновлению и исправлению уязвимостей;

Возможность отслеживания состояния безопасности образов в Docker Hub и Docker Desktop.

Основные возможности Docker Scout CLI



Пример команды для сканирования:

```
docker scout cves <image_name>
```

Использование Trivy и Docker Scout позволяет значительно повысить безопасность контейнеров, определять уязвимости еще на этапе разработки и тем самым предотвращать их появление в рабочей среде.

Проверка целостности образов и использование Docker Content Trust

Docker Content Trust (DCT) используется для подписывания образов и проверки подлинности перед развертыванием, что гарантирует их неизменность и защиту от подделок. Использование цифровой подписи в процессе разработки и развертывания образов обеспечивает надежную проверку, целостность и аутентичность контейнеров.

Пример включения Docker Content Trust:

```
export DOCKER_CONTENT_TRUST=1
```

Код выше позволит Docker проверять подписи при каждой операции `docker pull` и `docker push`, что минимизирует риск использования измененных или поддельных образов.

Управление секретами и конфиденциальностью

Когда работа с контейнерами связана с передачей конфиденциальных данных (паролей, токенов, ключей API), крайне важно правильно управлять ими. Для этого будет полезен Docker Secrets.

Docker Secrets

Механизм безопасного управления секретами в Docker используется для хранения и передачи конфиденциальных данных в контейнерные сервисы. Секреты в Docker хранятся в зашифрованном виде и передаются только тем контейнерам, которые явно запрашивают их использование. Они не записываются на диск, что исключает их утечку через файловую систему. Docker Secrets применяются в связке с Docker Swarm и позволяют задавать контроль доступа к конфиденциальным данным на уровне сервисов и задач.

Пример команды для создания секрета:

```
docker secret create db_password my_password.txt
```

После создания секрет можно привязать к контейнеру, указав его в конфигурации при развертывании:

```
docker service create --name my_service --secret db_password my_image
```

Таким образом получится исключить хранение секретов в переменных окружения окружения, которые могут быть случайно или намеренно считаны другими

пользователями или процессами.

Краткие рекомендации по интеграции лучших практик безопасности в DevSecOps

Чтобы обеспечить максимальный уровень защиты, важно интегрировать методы безопасности на всех этапах жизненного контейнеров — от создания образа до его развертывания и эксплуатации.

Автоматизируйте безопасность на всех этапах CI/CD. Интегрируйте инструменты статического и динамического анализа уязвимостей в пайплайны CI/CD, настройте автоматическое сканирование образов и применение политик безопасности.

Ограничивайте ненужные возможности. Используйте минималистичные базовые образы и принцип наименьших привилегий.

Настраивайте строгие политики доступа в Kubernetes. Задайте четкие границы между сервисами и установите ограничения на использование привилегированных контейнеров.

Управляйте секретами и конфиденциальностью: используйте Docker Secrets, настройте TLS для межконтейнерного взаимодействия и шифрование сетевого трафика.

Производите непрерывный мониторинг и аудит. Используйте инструменты для мониторинга состояния контейнеров и проводите регулярный аудит конфигураций и политик безопасности.

Улучшение безопасности DevSecOps



Итак, друзья, теперь, когда мы улучшили наши знания о защищенности контейнеров, пора действовать! Настройте свои политики, используйте лучшие практики и делайте безопасность неотъемлемой частью вашего DevSecOps-процесса. Помните, в мире контейнеров шутки плохи: чем сильнее защита, тем меньше шансов на успешный хак!